

EventHub API interview guide

Manual cases, Postman inputs, and the matching Playwright Java tests.

API: <https://api.eventhub.rahulshettyacademy.com/api/docs/>

How to run this in Postman

1. Open Postman. Click Import. Choose EventHub-Interview.postman_collection.json from Downloads.
2. Click the collection EventHub interview, then click Run.
3. Leave every folder ticked. Run from EH-01 through the delete requests.
4. Do not press Send on a later request by itself. It needs the email, token, and ids from earlier requests.
5. Green means the step passed. Open a red step to see which check failed.

Base URL: <https://api.eventhub.rahulshettyacademy.com/api>

The first request sets email to learner.<time>@example.com, password to secret12, and marker to case-<time>.

Protected calls use header Authorization: Bearer <token>.

How to run the same cases in Java

Project: Playwright-Java-API-Testing-main

Command: mvn -DsuiteXmlFile=src/test/resources/interview-testng.xml test

Package: com.qa.api.eventhub.interview

AuthenticationTest covers EH-01 to EH-06.

AuthorizationTest covers EH-07 and EH-08.

EventsTest covers EH-09 to EH-15. The Java update uses its own event so the list events stay unchanged.

BookingsTest covers EH-16 to EH-23.

PaginationFilterSortTest covers EH-24 to EH-32 and EH-35 to EH-38.

ConcurrencyTest covers EH-33 and EH-34.

Rules that confuse people

data: [] with success true means the call worked and no row matched. It is not always a failure.

GET /bookings is empty before you create a booking, and empty again after cancel. Cancel deletes the row.

A GET does not use a body. Creating a booking must be POST.

Booking fields are customerName, customerEmail, customerPhone. Not name, email, or phone. eventId and quantity are numbers. Do not put them in quotes.

Password must be at least 6 characters. secret12 is valid. abc is not.

Event date must be in the future. These cases use 2027-12-01T10:00:00.000Z.

sort=price does not reorder events. The newest event stays first.

Two bookings for the last seat should be one success and one rejection. This server often accepts both. Seats must not go below 0.

1 Authentication

EH-01 POST /auth/register. Input: email learner.<time>@example.com, password secret12.

Expected: 201, success true, a token, and the same email.

EH-02 POST /auth/login. Input: the same email and password secret12. Expected: 200 and a token.

EH-03 GET /auth/me. Input: Authorization Bearer token. Expected: 200 and the same email.

EH-04 POST /auth/register again. Input: the same email and password. Expected: 400, error Email already registered.

EH-05 POST /auth/register. Input: email short.<marker>@example.com, password abc. Expected: 400 Validation failed, details include field password.

EH-06 POST /auth/register. Input: email not-an-email, password secret12. Expected: 400, details include field email.

2 Authorization

EH-07 GET /events. Input: no token. Expected: 401, error Unauthorized.

EH-08 GET /auth/me. Input: Authorization Bearer not-a-token. Expected: 401, error Invalid or expired token.

3 Events

Three events are created. Titles contain the marker.

Workshop: category Workshop, city Chennai, venue Hall A, description desc-<marker>, price 10, seats 5. Id saved as eventId1.

Concert: category Concert, city Chennai, venue venue-<marker>, description music night, price 40, seats 5. Id saved as eventId2.

Sports: category Sports, city Hyderabad, venue Stadium, description match day, price 90, seats 5. Id saved as eventId3.

EH-09 POST /events. Input: { title: Incomplete } plus the token. Expected: 400 Validation failed.

EH-10 POST /events. Input: the workshop. Expected: 201, availableSeats 5, id saved as eventId1.

EH-11 GET /events/{{eventId1}}. Expected: 200, city Chennai.

EH-12 GET /events/99999999. Expected: 404.

EH-13 PUT /events/{{eventId1}}. Input: { title: Only title }. Expected: 400 Validation failed.

EH-14 POST /events. Input: the concert. Expected: 201, id saved as eventId2.

EH-15 POST /events. Input: the sports event. Expected: 201, id saved as eventId3.

Java EventsTest also replaces one separate event (venue Hall B, price 15) and deletes it twice. First delete is 200 Event deleted successfully. Second delete is 404.

4 Bookings

EH-16 GET /bookings?eventId={{eventId1}}. No body. Expected: 200, total 0, data empty. Correct, because no ticket exists yet.

EH-17 POST /bookings. Input: eventId eventId1 with no quotes, customerName Priya Sharma, customerEmail priya.sharma@email.com, customerPhone 9876543210, quantity 1 with no quotes. Expected: 201, status confirmed, totalPrice 10.

EH-18 GET /events/{{eventId1}}. Expected: availableSeats 4.

EH-19 GET /bookings?eventId={{eventId1}}. Expected: total 1 and the saved booking id. Look here to see the booking. Do this before cancel.

EH-20 GET /bookings/{{bookingId}}. Expected: 200, quantity 1.

EH-21 POST /bookings. Input: quantity 10 while 4 seats remain. Expected: 400 and the error contains available. Seats stay 4.

EH-22 DELETE /bookings/{{bookingId}}. Expected: 200, message Booking cancelled.

EH-23 GET /bookings?eventId={{eventId1}}. Expected: data empty, total 0. Then the event has availableSeats 5.

5 Pagination

All three calls use search={{marker}} and limit=2. Only the 3 events above match.
EH-24 page=1. Expected: 2 rows, total 3, page 1, limit 2, totalPages 2.
EH-25 page=2. Expected: 1 row. That id is not on page 1. total stays 3.
EH-26 page=3. Expected: 0 rows, total still 3, totalPages still 2. Empty data is correct.

6 Filtering

EH-27 search={{marker}} and category=Workshop. Expected: total 1, the workshop.
EH-28 search={{marker}} and city=chennai. Lowercase still matches. Expected: total 2, both city Chennai.
EH-29 search=desc-{{marker}}. Expected: 1 row, the workshop. Search checks title, description, and venue.
EH-30 search={{marker}}, category=Sports, city=Chennai. Expected: data empty, total 0. The sports event is in Hyderabad.

7 Sorting

EH-31 GET /events?search={{marker}}&limit=10. Expected order: sports, concert, workshop. Prices 90, 40, 10. The sports event was created last.
EH-32 Add sort=price and order=asc. Price low to high would be workshop 10, concert 40, sports 90. Expected: sports, concert, workshop and prices 90, 40, 10. The API does not sort by price.
EH-35 Same search with sort=price and order=desc. Switching asc to desc does not flip the list. Expected: sports, concert, workshop again.
EH-36 sort=title and order=asc. Title A to Z would be concert, sports, workshop. Expected: sports, concert, workshop.
EH-37 sort=city and order=asc. City A to Z would start with Chennai. Expected: first row is Hyderabad, then Chennai, then Chennai.
EH-38 sort=title and order=asc with page=1 and limit=2, then page=2. Page 1 is sports and concert. Page 2 is the workshop. Each id is smaller than the id before it.

8 Concurrent calls

EH-33 Register one.<time>@example.com and two.<time>@example.com at the same time. Password secret12 for both. Expected: both return 201.
EH-34 Create an event with totalSeats 1 and price 80. Send two POST /bookings with quantity 1 at the same time. Customer is Priya Sharma, priya.sharma@email.com, phone 9876543210. Expected for a correct seat lock: one response 201 and the other 400. availableSeats ends at 0.
What this server often does: both responses are 201, and two confirmed tickets exist for one seat. The automated check allows 201 or 400 and fails if seats go below 0.

Cleanup

DELETE /events/{{eventId1}}, then eventId2, then eventId3. Each expected result is 200.
Run cleanup only after the earlier folders. A new collection run creates new ids.